Presentazione di

DietiEstates25

Università degli Studi di Napoli Federico II

A cura del team INGSW2425_055

Visconti Gaetano

N86002282

Borgia Manuele

N86003557

Obiettivi richiesti – le funzionalità chiave

Si richiede lo sviluppo di una **piattaforma per la gestione di servizi immobiliari** che concentri la sua attenzione sui seguenti requisiti:

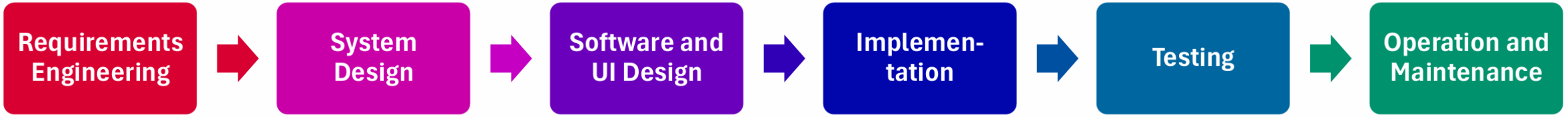
- **Autenticazione e gestione degli utenti** (Req. 1)
- **Caricamento** (da parte degli Agenti) **di nuovi immobili** (Req. 2)
- **Ricerca** (da parte dei Clienti) **di nuovi immobili fra quelli presenti** (Req. 3)
- **Interazione Cliente-Agente con offerte sugli immobili** (Req. 8)
- **Localizzazione dell'immobile in prossimità di servizi** (Req. 9)

L'obiettivo è di ottenere un **prodotto software** professionale e, in quanto non immediato da risolvere, necessita di un **processo** ed un approccio più articolato.

Strategia adottata – il modello «waterfall»

Tale modello è un **processo sequenziale** e serve al team di sviluppo **per decomporre il processo software** in moduli identificati da fasi distinte e non sovrapposte.
Si giustifica la scelta di tale modello con le seguenti caratteristiche:

- **Compattezza:** Ogni fase produce un output autosufficiente per le fasi successive.
- **Adatto al contesto:** Team formato da 2 persone.

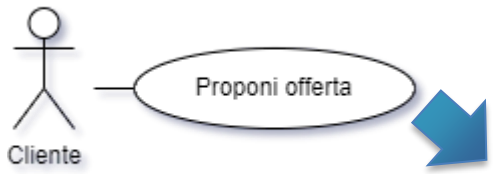


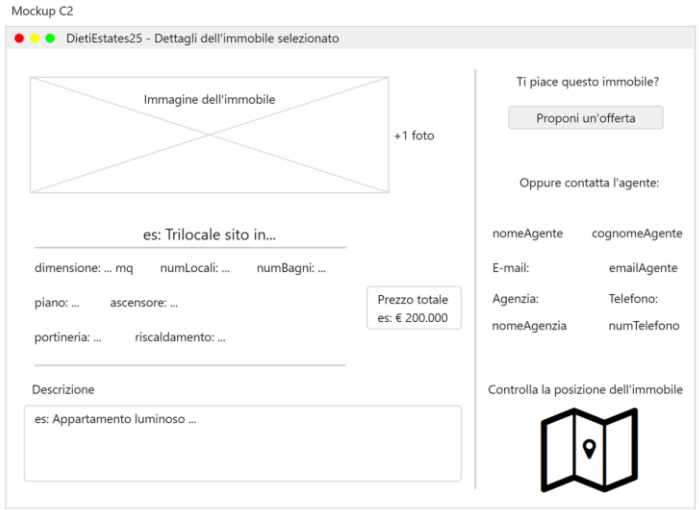
Il modello a cascata.
Fonte immagine: diapositive del corso

Analisi dei requisiti

Questa fase produce il **documento dei requisiti**, esso raccoglie in linguaggio naturale le interazioni tra l'analista ed i committenti.
In questo documento, una volta ultimato, troveremo informazioni su:

- **Target Utente (Personas):** Identificazione di 4 Personas (es. Maria Rossi, Giampiero Torre) per rappresentare le esigenze di clienti e agenti.
- **Modellazione UML:** Realizzazione di Use Case Diagram per utente non registrato, cliente, agente/admin.
- **Requisiti Non Funzionali e di Dominio:** Per garantire usabilità e regole di business (es. RD-02: offerta iniziale < prezzo annuncio).
- **Prototipazione:** Mockup a bassa fedeltà e tabelle di Cockburn per casi d'uso chiave





USE CASE #9	Proponi offerta
Obiettivo	Il Cliente vuole proporre un'offerta per un immobile
Precondizioni	Il Cliente ha effettuato l'accesso all'applicativo; Il Cliente si trova sulla Home MC1; Il Cliente ha già effettuato una ricerca di immobili e ne ha individuato uno di suo interesse
Condizione di successo	Il Cliente riesce a comunicare all'agente immobiliare l'interesse per l'immobile riuscendo a proporre un'offerta

System design

Questa è la fase in cui «diventiamo» **Software Architect**, qui si prendono le scelte più importanti per stabilire le fondamenta architetturali del prodotto da sviluppare, individuati i requisiti software si comincia a definire:

- **Architettura Logica:** Pattern **MVC (Model-View-Controller)** per separare la logica di business, presentazione e controllo garantendo alta coesione e basso accoppiamento.

- **Distribuzione:** Architettura distribuita su cloud (**AWS**).
  Il front-end è indipendente dal back-end.

- **Persistenza:** Database **PostgreSQL** su AWS RDS. Schema logico che modella la gerarchia degli account (Account ← Cliente/Agente) e la gestione delle offerte (OffertaIniziale - RispostaOfferta).

- **Motivazioni:** Disponibilità, scalabilità, manutenibilità, sicurezza e indipendenza dai servizi esterni (es. Cognito è impiegato per la sola autenticazione).

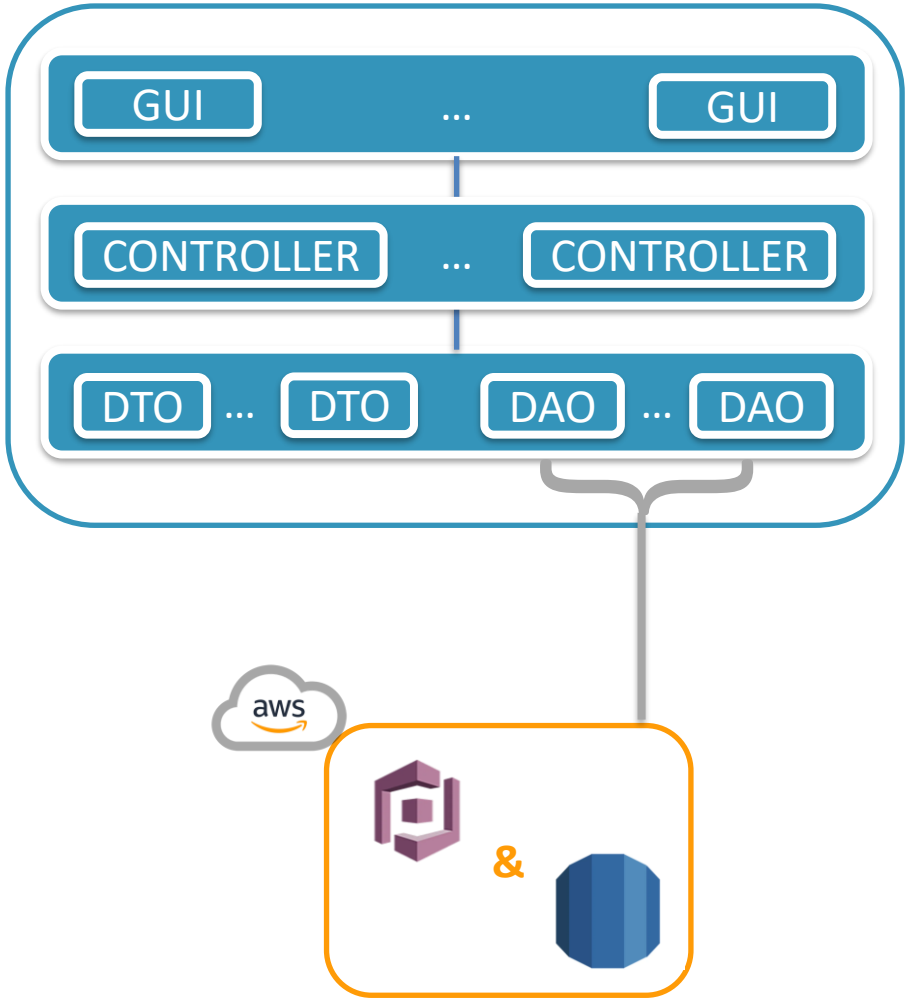


Illustrazione dell'architettura adottata

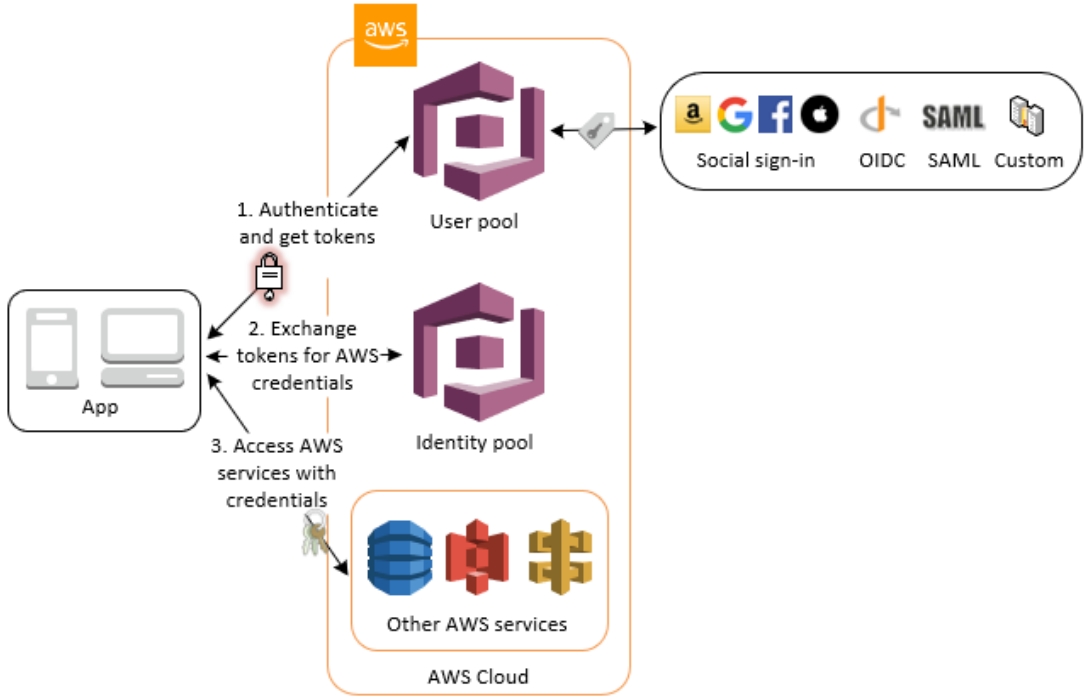
AWS e i servizi impiegati

- **Amazon RDS (PostgreSQL):** Database gestito ad alta disponibilità, esso semplifica la configurazione, l'operatività e la scalabilità di un database relazionale nel cloud. Verrà scelto PostgreSQL per la sua robustezza e supporto a documenti come JSON (per i filtri di ricerca).

- **Amazon Cognito:** Deleghiamo la gestione di autenticazione e registrazione. Si gestiranno solo i dati anagrafici nel DB (nella tabella Account).

- **API Places (Google):** Geolocalizzazione su mappa che, mediante un browser, consente all'utente di vedere i servizi vicini ad un immobile. (Req. 9)

- **Indipendenza:** L'uso di servizi esterni è limitato a funzionalità specifiche.



L'immagine descrive il flusso di autenticazione e autorizzazione per permettere ad un utente di accedere in modo sicuro ai servizi AWS tramite Amazon Cognito.

Fonte immagine: Amazon Web Services

Object design

Questa fase dello sviluppo software guarda alla conclusione del **documento di design del sistema**, esso sarà prevalentemente composto da diagrammi UML e sarà utile per la successiva fase di implementazione del codice, la parte di **object design** prevede:

- **Diagramma delle Classi di Design:** Diagrammi UML che descrivono in modo tecnico i casi d'uso, integrando il pattern MVC con il pattern **DAO** per la persistenza.
- **Diagrammi di Sequenza:** Modellato il flusso dinamico per gli stessi casi d'uso, mostrando l'interazione temporale tra oggetti (View ↔ Controller ↔ Model ↔ DAO).
- **Responsabilità Chiare:** Ogni classe ha una singola responsabilità (es. ImmobileDAO gestisce solo l'accesso ai dati sugli immobili).

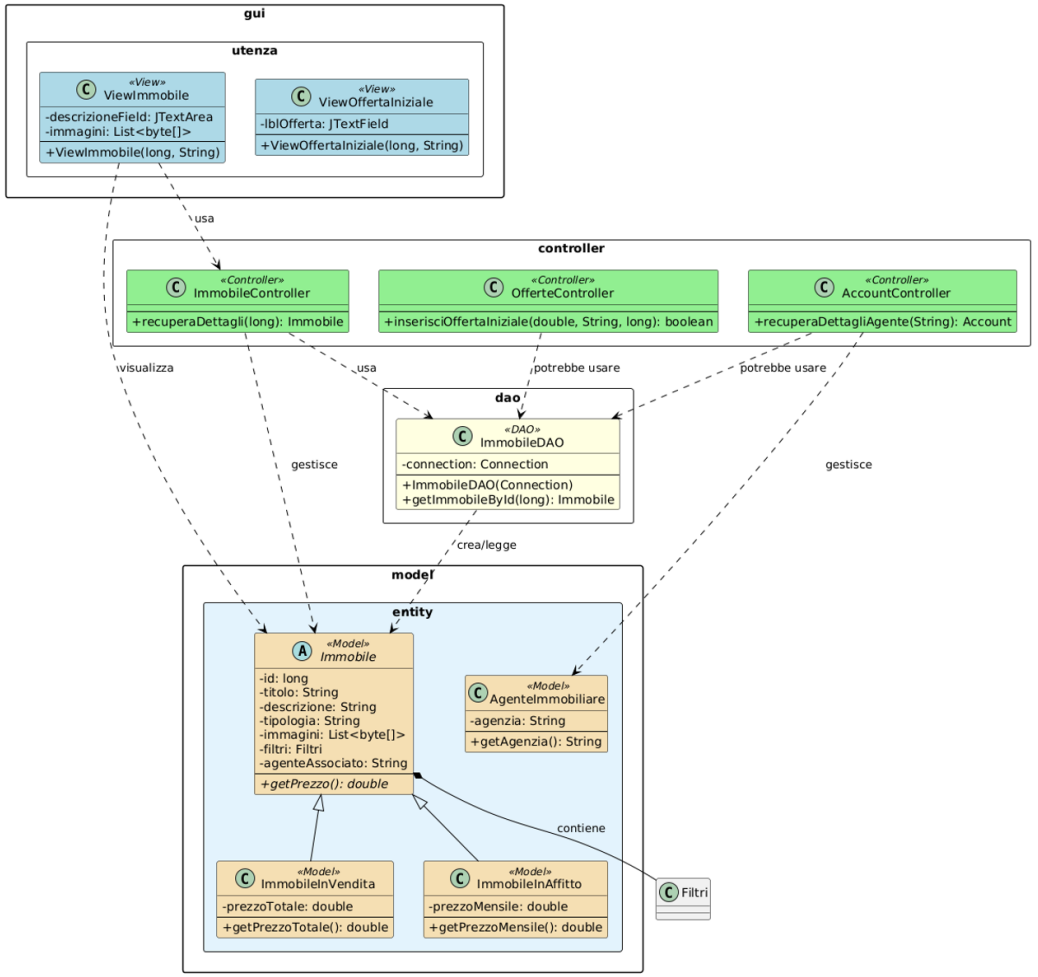


Diagramma delle classi che descrive come un cliente propone un'offerta

Implementazione e qualità del codice

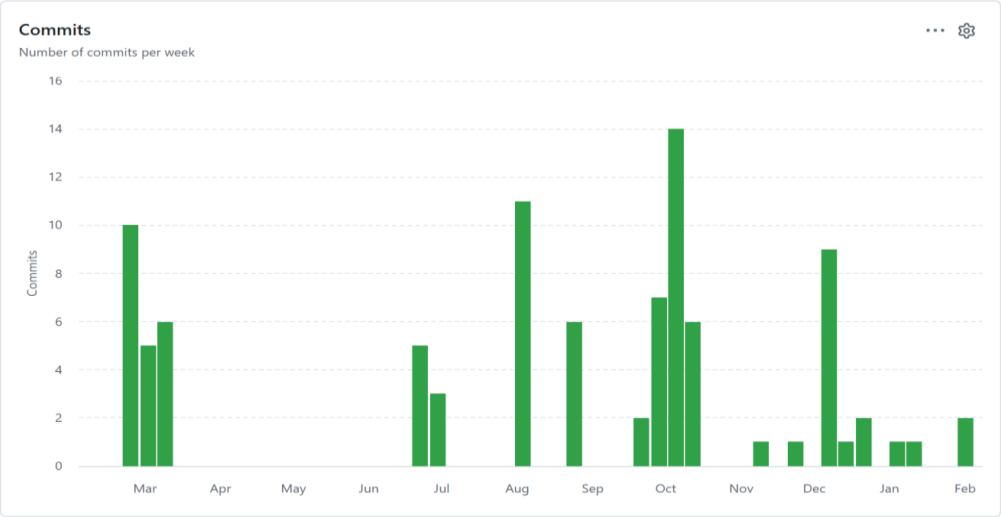
- **Tecnologie:** Java 21, Maven per dipendenze e build automation, OkHttp per richieste HTTP, JWT per token, jBCrypt per hashing.

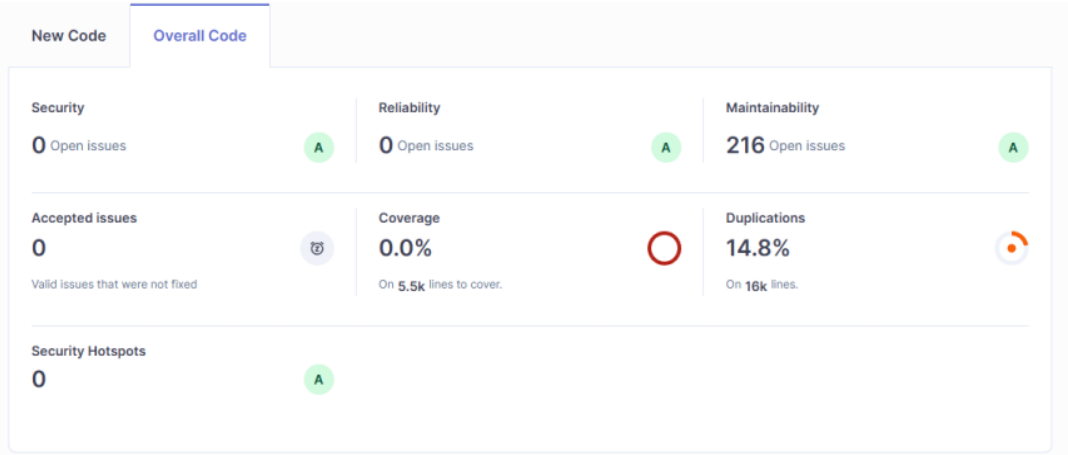
- **Versioning (Git):** Ospitato su GitHub con ~100 commit, 2 contributors, attività distribuita su tutto l'arco del progetto. Commit frequenti e significativi.

- **Quality Assurance (SonarQube):** Integrato nel processo di build con Maven (mvn clean verify sonar:sonar). Analisi statica continua per identificare bug, vulnerabilità e code smells.

Risultato: Codice distribuito, pulito ed affidabile.

Commits over the last year of **g-visconti/CodProgettoIngSW055**





Testing

- **Framework: JUnit 5** (xUnit per Java) per Unit Testing.

- **Strategia di Progetto:** Black-box testing con partizionamento in classi di equivalenza e analisi dei casi limite.

- **Metodi Testati (4):**
  - isValidEmail(String email, boolean allowEmpty) - 5 CE
    isValidPassword(String password, String conferma) - 4 CE
  - validaImmobile(...) (multiparametro) - 5 CE (es. campo vuoto, prezzo zero).
  - isValidControproposta(double controproposta, double offertaIniziale) - 5 CE (es. controproposta < offerta, negativa).

Risultato: Copertura delle principali combinazioni di input e validazione del corretto comportamento.

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running controllertest.AccessControllerTest
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.120 s
Running controllertest.ImmobileControllerTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.052 s

Running controllertest.OfferteControllerTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.027 s


Results:

Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
```

Output che si ottiene eseguendo i test da terminale inserendo la stringa «mvn clean test»

Validazione del prodotto software

- **Expert Review (Checklist):** Applicata una checklist basata sulle euristiche di Nielsen. Risultati positivi su feedback, coerenza e prevenzione errori.

- **Esperimento con Utenti Reali (4):** 2 clienti inesperti, 1 cliente esperto, 1 agente immobiliare.

- **Procedura:** Task definiti (registrazione, ricerca, logout) e metriche raccolte (tempo, errori, completamento).

Risultati:

- **Apprendibilità:** Utenti inesperti mostrano una curva di apprendimento rapida. (Tutti partono da zero).
- **Soddisfazione:** Survey post-task con una media voti > 4 per l'intero questionario

Il grafico mostra l'andamento dell'apprendibilità del sistema durante l'esecuzione dei principali task

Strumenti impiegati

- **Modellazione & Prototipazione:** draw.io (UCD), Figma (Personas), mydraft.cc (Mockup), StarUML (Classi), Visual Paradigm (Sequence), Canva (Grafici).
- **Cloud & Infrastruttura:** AWS (RDS, Cognito), Docker, PgAdmin.
- **Sviluppo & Build:** Eclipse, Java 21, Apache Maven, GitHub/GitHub Desktop.
- **Qualità & Testing:** JUnit 5, SonarQube/SonarLint.
- **Documentazione e presentazione:** Pacchetto Microsoft 365.

Grazie per l’attenzione